Task 1

```python
    if not stamped and frame_id:
        raise Exception("'frame_id' can only be set when 'stamped' is True")

    if stamped:
        TwistMsg = geometry_msgs.msg.TwistStamped
    else:
        TwistMsg = geometry_msgs.msg.Twist

    pub = node.create_publisher(TwistMsg, 'velocity', 10) #Andrew Ebinger changed topic name from "cmd_vel" to "velocity" on 3/13/2025 at 11:03 AM to satisfy the requirements of his IRP lab assignment

    spinner = threading.Thread(target=rclpy.spin, args=(node,))
    spinner.start()

```

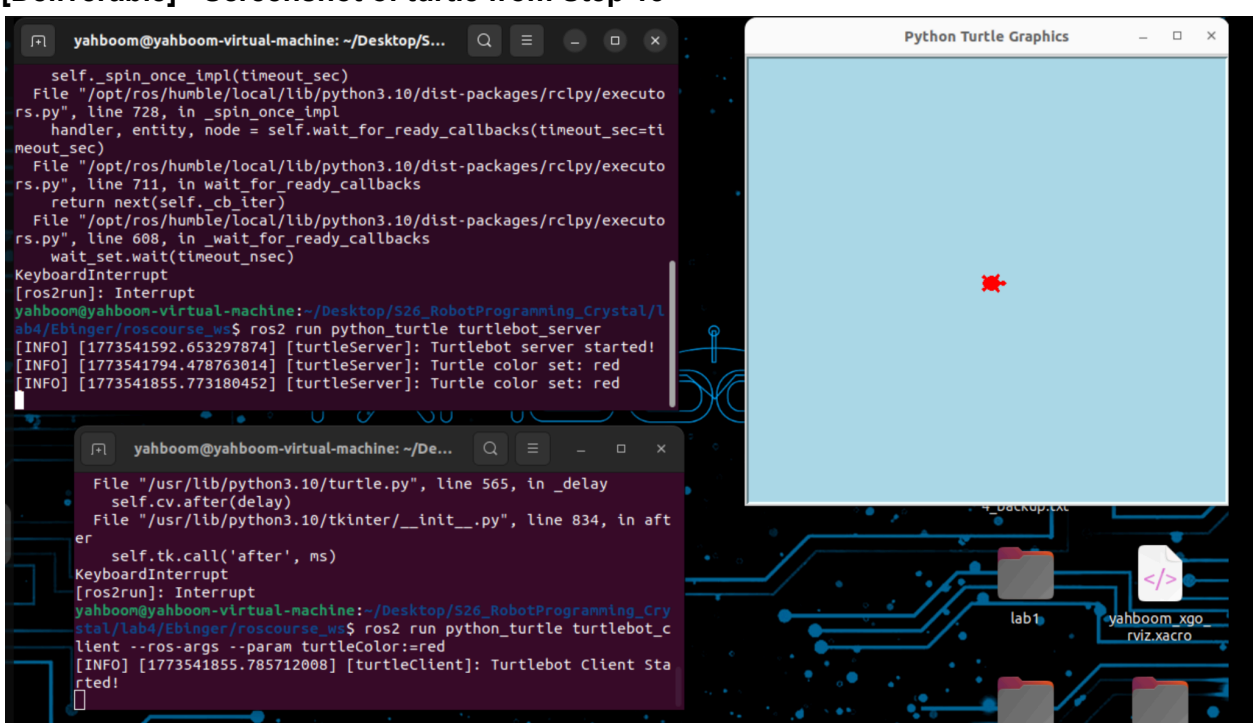
[Deliverable] - Screenshot of terminals from Step 17 + answer from question

Question 17 - When trying to move the turtle with the keyboard, I found myself unable to do so. Possible reasons why are:

- The topic names in the publisher command for teleop_keyboard do not line up with the receiving node names
- I commented out too many lines in the python turtle package trying to stop the message being sent.
- Actual reason was that the default speed is too slow to be seen, and needed to be adjusted.

Task 2

[Deliverable] - Screenshot of turtle from Step 13



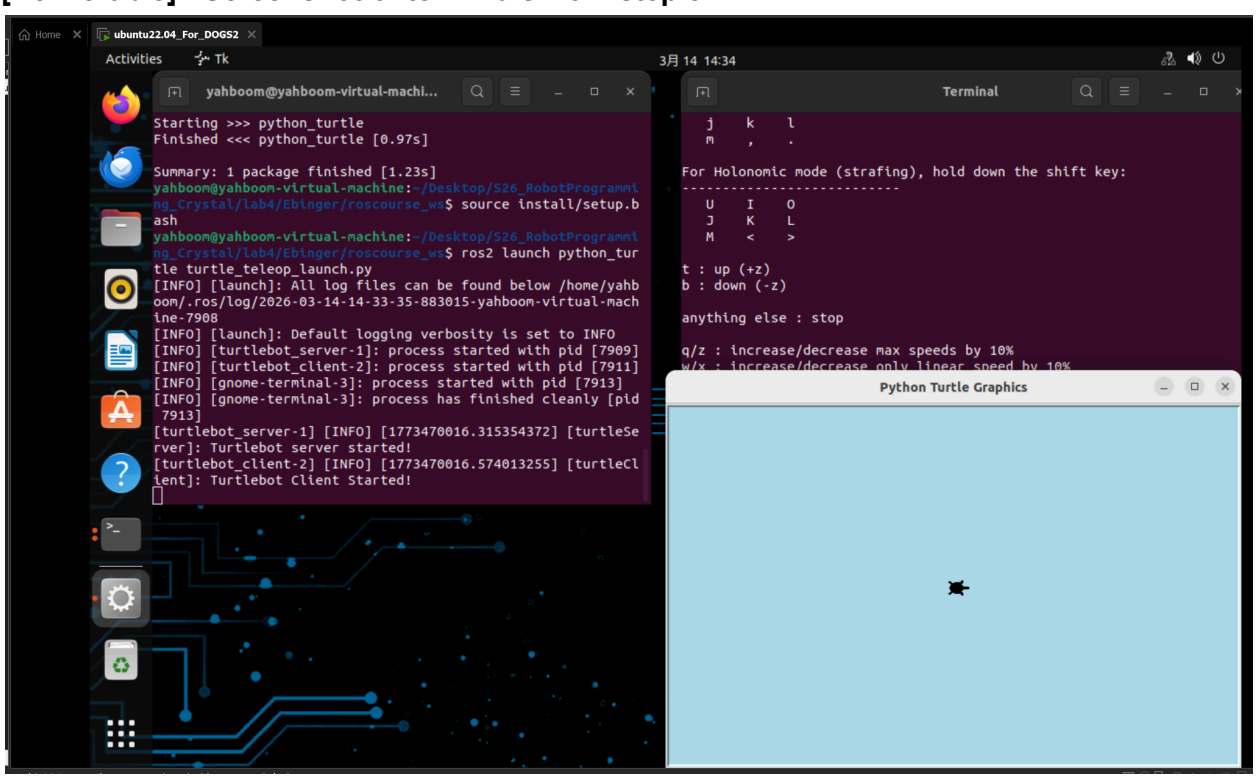
[Deliverable] - Modified turtlebot_client.py with comments and formatting from Step 14

turtlebot_client.py
~/Desktop/S26_RobotProgramming_Crystal/l...ourse_ws/src/python_turtle/python_turtle

```python
        self.turtle_display.shape("turtle")
        self.turtle = TurtleMsg()

        #### publisher define ####
        self.twist_pub = self.create_publisher(Twist, 'turtleDrive', 1)
        ##########################

        #### subscribing turtlebot state ####
        self.turtle_sub = self.create_subscription(TurtleMsg, 'turtleState', self.turtle_callback, 1)

        #### parameter for setting color ####
        self.declare_parameter('turtleColor', 'blue', ParameterDescriptor(description='Defines the color of the turtle [not the trail] on startup.'))

        #### reads parameter value and sets the turtle to that color ####
        turtleColor = self.get_parameter('turtleColor').get_parameter_value().string_value
        self.turtle_display.color(turtleColor)

        #### allocates space for color parameter to be read with a service call ####
        self.color_cli = self.create_client(SetColor, 'set_Color')
        while not self.color_cli.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('Color service not available, waiting...')
        self.color_req = SetColor.Request()
        self.color_req.color = turtleColor
        self.server_call = True
        self.service_future = self.color_cli.call_async(self.color_req)

        #### parameter for setting pen size ####
        self.declare_parameter('penSize', 2, ParameterDescriptor(description='Defines the pen thickness of the turtle trail on startup.'))

        #### reads parameter value and sets the turtle pen size ####
        penSize = self.get_parameter('penSize').get_parameter_value().integer_value
        self.turtle_display.pensize(penSize)

```

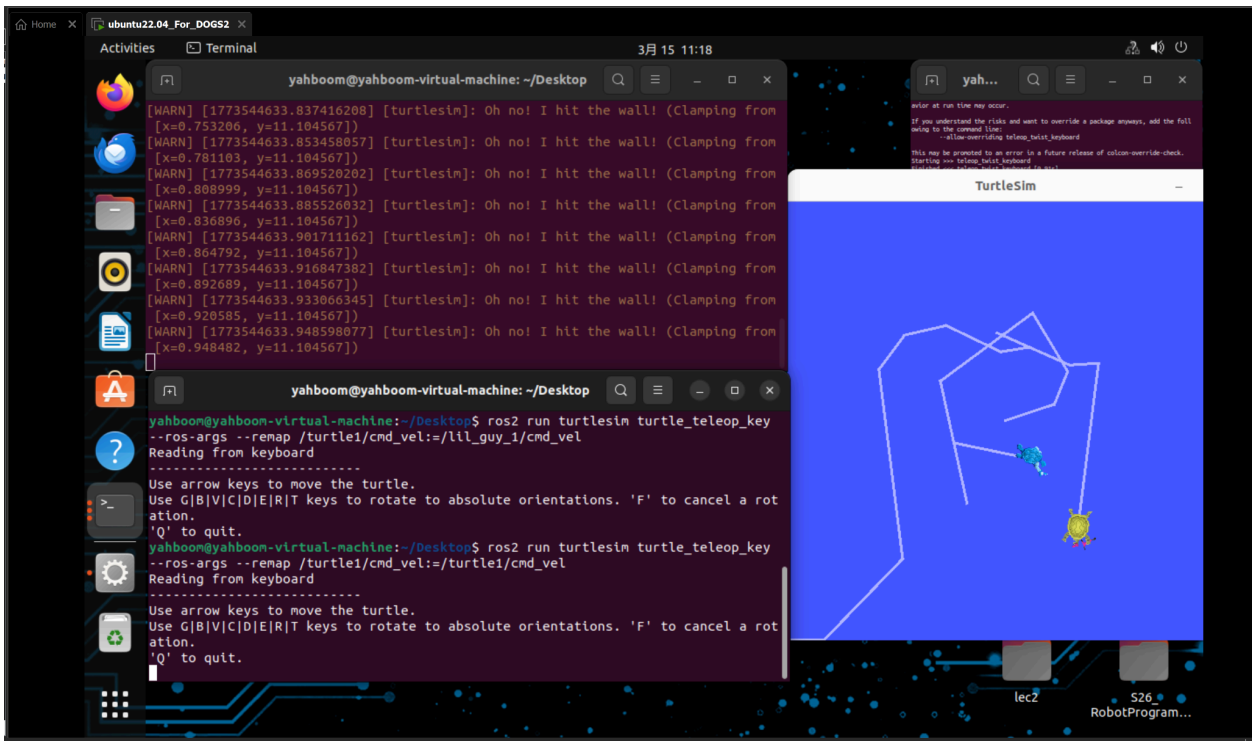
Task 3

[Deliverable] - Screenshot of terminals from step 8



Task 4

[Deliverable] - Screenshot of the window with multiple turtles that have moved with different colors as well as a screenshot (one combined is fine) showing the terminals performing remote control.



Required question: Remapping is useful because it saves a great deal of time during testing of individual portions of ones code. Instead of having to navigate all the way down the tree to change a parameter, I am able to simply change it during the command line call.